ARTIFICIAL INTELLEGENCE AND DEEP LEARNING

LAB ASSIGNMENT – 10

NAME – SUFIYA AFREEN

ENROLLMENT NO – 2403A51067

BRANCH/BATCH – CSE-03

YEAR/SEM – 3-1

DATE – 16-09-26

Lab/Exp 10: Transfer Learning: Fine-tuning a Pre-trained CNN

Kaggle Dataset Link: https://www.kaggle.com/datasets/ Cat and Dog

1. Load and preprocess the Cats vs Dogs (ResNet18) dataset.

2. Implement the required PyTorch model.

3. Train using suitable hyperparameters.

4. Evaluate using appropriate metrics.

5. Visualize results (loss/accuracy/features as applicable).

6. Summarize observations and conclusions.

CODE & OUTPUT –

```python
import torch
import torch.nn as nn
import torch.optim as optim

from torchvision import datasets, transforms, models
from torch.utils.data import DataLoader, random_split

import matplotlib.pyplot as plt
import numpy as np

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    ConfusionMatrixDisplay
)

import os

# Device
device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

print("Device:", device)

transform = transforms.Compose([

    transforms.Resize((224, 224)),

    transforms.RandomHorizontalFlip(),

    transforms.ToTensor(),

    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )

])
```

```python
dataset = datasets.OxfordIIITPet(
    root="./data",
    split="trainval",
    target_types="category",
    transform=transform,
    download=True
)

print("Total Images:", len(dataset))

print("Classes:", dataset.classes)

train_size = int(0.8 * len(dataset))

test_size = len(dataset) - train_size

train_dataset, test_dataset = random_split(
    dataset,
    [train_size, test_size],
    generator=torch.Generator().manual_seed(42)
)

train_loader = DataLoader(
    train_dataset,
    batch_size=32,
    shuffle=True,
    num_workers=2
)

test_loader = DataLoader(
    test_dataset,
    batch_size=32,
    shuffle=False,
    num_workers=2
)

print("Training Images:", len(train_dataset))

print("Testing Images:", len(test_dataset))

images, labels = next(iter(train_loader))

plt.figure(figsize=(10, 6))

for i in range(8):
```

```
        plt.subplot(2, 4, i + 1)

        image = images[i].permute(1, 2, 0).numpy()

        mean = np.array([0.485, 0.456, 0.406])

        std = np.array([0.229, 0.224, 0.225])

        image = image * std + mean

        image = np.clip(image, 0, 1)

        plt.imshow(image)

        plt.title(
            "Cat" if labels[i].item() == 0 else "Dog"
        )

        plt.axis("off")

    plt.tight_layout()

    plt.show()

    weights = models.ResNet18_Weights.DEFAULT

    model = models.resnet18(weights=weights)

    # Freeze all pre-trained layers
    for param in model.parameters():

        param.requires_grad = False


    model.fc = nn.Linear(
        model.fc.in_features,
        2
    )

    model = model.to(device)

    print("\nModel:")
    print(model)

    criterion = nn.CrossEntropyLoss()
```

```
optimizer = optim.Adam(
    model.fc.parameters(),
    lr=0.001
)

num_epochs = 5


train_losses = []

train_accuracies = []

print("\nTraining Started...\n")

for epoch in range(num_epochs):

    model.train()

    running_loss = 0

    correct = 0

    total = 0

    for images, labels in train_loader:

        images = images.to(device)

        labels = labels.to(device)

        # Clear gradients
        optimizer.zero_grad()

        # Forward pass
        outputs = model(images)

        # Calculate loss
        loss = criterion(outputs, labels)

        # Backward pass
        loss.backward()

        # Update weights
        optimizer.step()
```

```
            running_loss += loss.item() * images.size(0)

            _, predicted = torch.max(outputs, 1)

            total += labels.size(0)

            correct += (
                predicted == labels
            ).sum().item()

        epoch_loss = running_loss / total

        epoch_accuracy = 100 * correct / total

        train_losses.append(epoch_loss)

        train_accuracies.append(epoch_accuracy)

        print(
            f"Epoch [{epoch+1}/{num_epochs}] "
            f"Loss: {epoch_loss:.4f} "
            f"Accuracy: {epoch_accuracy:.2f}%"
        )

    model.eval()

    all_labels = []

    all_predictions = []

    with torch.no_grad():

        for images, labels in test_loader:

            images = images.to(device)

            outputs = model(images)

            _, predicted = torch.max(outputs, 1)

            all_labels.extend(
                labels.numpy()
            )

            all_predictions.extend(
```

```
            predicted.cpu().numpy()
        )


# Calculate metrics

accuracy = accuracy_score(
    all_labels,
    all_predictions
)

precision = precision_score(
    all_labels,
    all_predictions,
    zero_division=0
)

recall = recall_score(
    all_labels,
    all_predictions,
    zero_division=0
)

f1 = f1_score(
    all_labels,
    all_predictions,
    zero_division=0
)


print("\n==============================")

print("EVALUATION RESULTS")

print("==============================")

print(
    f"Accuracy  : {accuracy * 100:.2f}%"
)

print(
    f"Precision : {precision:.4f}"
)

print(
    f"Recall    : {recall:.4f}"
```

```
print(
    f"F1 Score  : {f1:.4f}"
)

plt.figure(figsize=(12, 5))


# Training Loss

plt.subplot(1, 2, 1)

plt.plot(
    range(1, num_epochs + 1),
    train_losses,
    marker="o"
)

plt.title("Training Loss")

plt.xlabel("Epoch")

plt.ylabel("Loss")

plt.grid()


# Training Accuracy

plt.subplot(1, 2, 2)

plt.plot(
    range(1, num_epochs + 1),
    train_accuracies,
    marker="o"
)

plt.title("Training Accuracy")

plt.xlabel("Epoch")

plt.ylabel("Accuracy (%)")

plt.grid()
```

```
plt.tight_layout()

plt.show()

cm = confusion_matrix(
    all_labels,
    all_predictions
)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Cat", "Dog"]
)

disp.plot(cmap="Blues")

plt.title("Confusion Matrix")

plt.show()


images, labels = next(iter(test_loader))

images_device = images.to(device)

model.eval()

with torch.no_grad():

    outputs = model(images_device)

    _, predictions = torch.max(outputs, 1)


plt.figure(figsize=(12, 6))

for i in range(8):

    plt.subplot(2, 4, i + 1)

    image = images[i].permute(1, 2, 0).numpy()

    mean = np.array([0.485, 0.456, 0.406])

    std = np.array([0.229, 0.224, 0.225])
```

```
        image = image * std + mean

        image = np.clip(image, 0, 1)

        plt.imshow(image)

        actual = (
            "Cat" if labels[i].item() == 0
            else "Dog"
        )

        predicted = (
            "Cat" if predictions[i].item() == 0
            else "Dog"
        )

        plt.title(
            f"Actual: {actual}\n"
            f"Predicted: {predicted}"
        )

        plt.axis("off")

    plt.tight_layout()

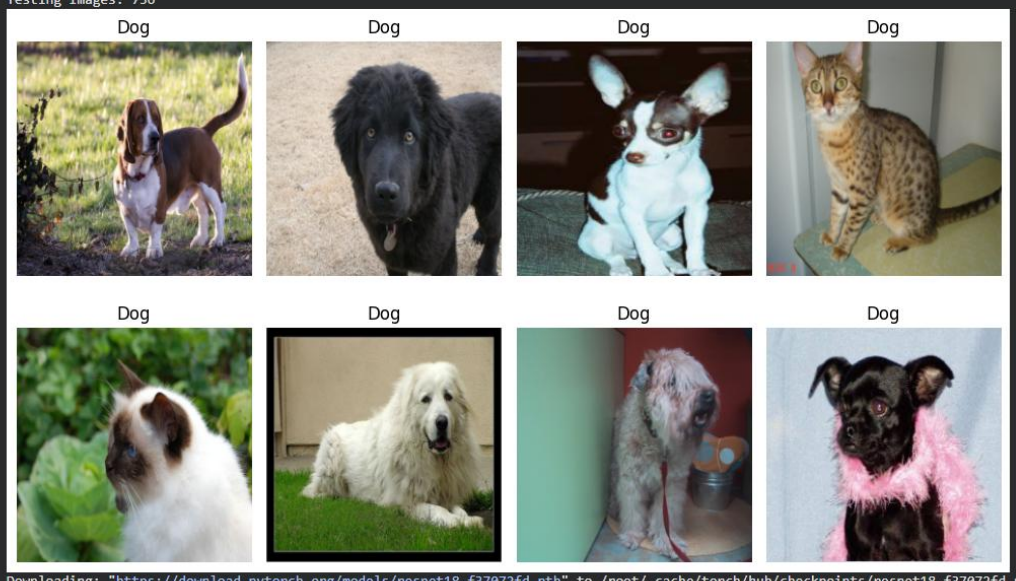
    plt.show()


    torch.save(
        model.state_dict(),
        "resnet18_cats_dogs.pth"
    )

    print("\nModel saved successfully!")
```

OUTPUT –

```
Device: cpu
100%|██████████| 792M/792M [00:24<00:00, 31.9MB/s]
100%|██████████| 19.2M/19.2M [00:01<00:00, 14.1MB/s]
Total Images: 3680
Classes: ['Abyssinian', 'American Bulldog', 'American Pit Bull Terrier', 'Basset Hound', 'Beagle', 'Bengal', 'Birman', 'Bombay', 'Boxer', 'British Sh
Training Images: 2944
Testing Images: 736
```




```
Downloading: "https://download.pytorch.org/models/resnet18-f37072fd.pth" to /root/.cache/torch/hub/checkpoints/resnet18-f37072fd.pth
100%|██████████| 44.7M/44.7M [00:00<00:00, 79.5MB/s]
```